

10. Modelos Fundamentais: Interação, Falhas e Segurança

Objetivos de aprendizagem

- Explicar modelos de interação síncrono e assíncrono.
- Classificar falhas de processo e comunicação.
- Relacionar timeout a detectores imperfeitos de falha.
- Analisar ameaças e controles de segurança em canais e serviços distribuídos.
- Executar experimentos controlados de falha.

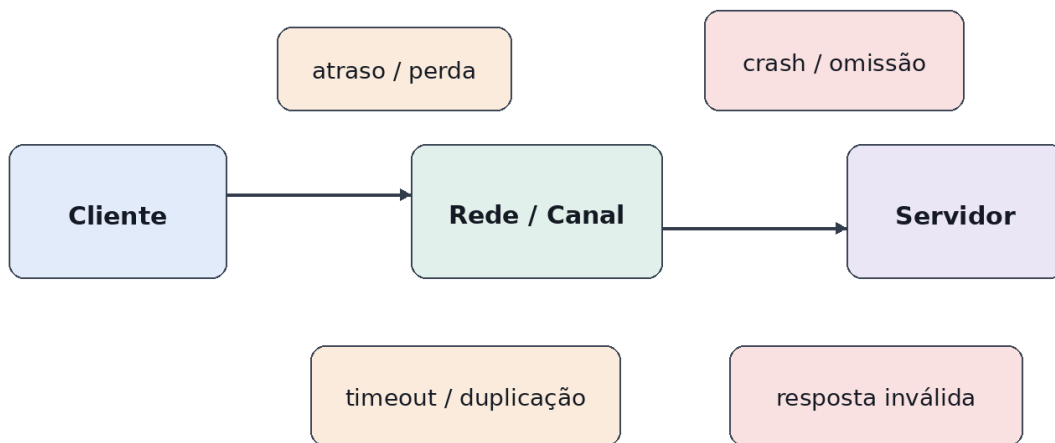


Figura 7 - Falhas podem ocorrer no processo, canal ou temporização

10.1 Modelo de interação

O modelo de interação descreve suposições sobre tempo de processamento, atraso de mensagens e deriva de relógio. Em um modelo síncrono, existem limites conhecidos para essas grandezas; em um modelo assíncrono, não há limite conhecido útil. Sistemas reais costumam operar em região intermediária: normalmente os atrasos são limitados, mas durante congestionamento, pausa de runtime, falha ou particionamento, podem exceder expectativas.

10.2 Por que timeout não prova falha?

Quando um cliente atinge o timeout, ele sabe apenas que não recebeu uma resposta dentro do prazo. O servidor pode ter falhado, a requisição pode ter sido perdida, o servidor pode estar lento, a resposta pode ter sido perdida ou apenas atrasada. Portanto, timeouts são mecanismos de decisão operacional baseados em suspeita, não oráculos perfeitos.

10.3 Falhas de processo e comunicação

Tabela 17 - Classificação didática de falhas

Classe	Descrição	Exemplo
Crash / fail-stop	Processo deixa de executar; em fail-stop a falha é detectável de modo confiável no modelo.	Servidor encerra por erro fatal.
Omissão de envio	Mensagem que deveria ser enviada não é enviada.	Buffer/erro impede publicação.

Classe	Descrição	Exemplo
Omissão de recepção	Mensagem chega ao host, mas não é processada/recebida.	Fila saturada ou socket fechado.
Temporização	Resposta ocorre fora do intervalo assumido.	Pausa longa ou congestionamento.
Resposta	Valor ou transição de estado incorretos.	Bug retorna cálculo errado.
Bizantina	Comportamento arbitrário, inclusive contraditório ou malicioso.	Nó envia valores diferentes a participantes.

10.4 Retries e backoff exponencial

Retries melhoram disponibilidade diante de falhas transitórias, mas retries imediatos por milhares de clientes podem agravar uma queda. Backoff exponencial aumenta progressivamente o intervalo e jitter adiciona aleatoriedade para evitar sincronização dos clientes. Circuit breaker é outro padrão: após várias falhas, o chamador evita novas tentativas por um período, permitindo recuperação do serviço.

10.5 Segurança: objetivos e ameaças

- Confidencialidade: impedir leitura por entidades não autorizadas.
- Integridade: detectar ou impedir alteração indevida.
- Autenticidade: confirmar identidade/origem.
- Autorização: limitar o que uma identidade pode fazer.
- Disponibilidade: manter serviço sob falhas e abuso.
- Não repúdio/auditoria: registrar evidências quando o domínio exige.

10.6 Canais seguros e identidade

TLS protege confidencialidade e integridade do canal e autentica pelo menos o servidor em configurações comuns. Isso não substitui autorização de aplicação. Um cliente autenticado pode continuar sem permissão para acessar determinado recurso ou tópico. Em sistemas distribuídos internos, mTLS pode autenticar ambos os lados, mas exige gestão de certificados e identidade de workload.

10.7 Laboratório de falhas com API

Adicione ao servidor REST uma rota experimental que introduz atraso controlado e outra que falha de forma probabilística. Nunca mantenha esse tipo de endpoint em produção; ele existe para tornar os modelos observáveis em ambiente de laboratório.

trecho para main.py

```
import random, time
from fastapi import HTTPException

@app.get("/experimento/instavel")
def instavel(atraso_ms: int = 0, proba_falha: float = 0.0):
    if atraso_ms > 0:
        time.sleep(atraso_ms / 1000)
    if random.random() < proba_falha:
        raise HTTPException(status_code=503, detail="falha injetada")
    return {"ok": True, "atraso_ms": atraso_ms}
```

retry.py

```
import random, time, requests

def get_com_retry(url, tentativas=4, timeout=0.5):
    for tentativa in range(tentativas):
        try:
            r = requests.get(url, timeout=timeout)
            r.raise_for_status()
            return r.json()
        except (requests.Timeout, requests.ConnectionError, requests.HTTPError) as exc:
            if tentativa == tentativas - 1:
                raise
            espera = (2 ** tentativa) * 0.2 + random.uniform(0, 0.1)
            print(f"falha {type(exc).__name__}; retry em {espera:.2f}s")
            time.sleep(espera)
```

AP5 - Experimentos com interação, falhas e segurança

Atividade prática

Use um dos sistemas construídos anteriormente e submeta-o a falhas controladas. A atividade avalia capacidade de classificar o comportamento observado segundo modelos fundamentais.

Objetivos da atividade

- Diferenciar timeout, erro de aplicação e conexão recusada.
- Classificar falhas com vocabulário técnico.
- Avaliar retries e backoff.
- Identificar pelo menos uma ameaça de segurança e aplicar um controle simples.

Requisitos obrigatórios

- Pelo menos quatro cenários de falha distintos.
- Coleta de duração, resultado e tipo de erro por tentativa.
- Um cenário com retry limitado e backoff.
- Um cenário demonstrando efeito adverso de retry ou operação não idempotente.
- Um controle de segurança aplicável ao laboratório: token, ACL, TLS local ou validação robusta.

Roteiro mínimo

1. Defina hipótese para cada falha.
2. Execute cenário de referência sem falha.
3. Introduza atraso, indisponibilidade e erro.
4. Aplique retry e compare.
5. Registre evidências e classifique cada caso.
6. Aplique e teste um controle de segurança.

Entregáveis

- Código/configuração de injeção de falhas.
- Tabela com hipótese, falha, observação, classificação e conclusão.
- Logs ou métricas.
- Breve análise de segurança.
- Demonstração de um cenário selecionado.

Questões para análise

1. O timeout permite afirmar que o servidor falhou?
2. Qual falha foi mais difícil de distinguir apenas pelo cliente?
3. Em qual cenário retry melhorou o resultado? Em qual piorou?
4. Que propriedade de segurança foi tratada pelo controle escolhido?
5. Como o mesmo sistema se comportaria se dois componentes falhassem simultaneamente?

Critérios de avaliação

Tabela 18 - Critérios de avaliação da AP5

Critério	Evidência esperada
Planejamento experimental	Hipóteses e cenários claros.
Execução	Falhas são realmente reproduzidas e medidas.
Classificação	Modelos fundamentais aplicados corretamente.
Resiliência	Retries/timeout analisados criticamente.
Segurança	Ameaça e controle coerentes.

Checklist antes da apresentação

- ☐ Existe cenário de referência.
- ☐ As falhas são reproduzíveis.
- ☐ Cada cenário possui evidência.
- ☐ Retries possuem limite e backoff.
- ☐ A análise distingue observação de inferência.

Desafios opcionais

- Use toxiproxy ou ferramenta equivalente para atraso/perda.
- Adicione métricas de percentis de latência.
- Implemente circuit breaker simples.

10.8 Relógios, ordenação e causalidade

Timestamps físicos ajudam auditoria e experiência do usuário, mas não fornecem uma ordem global perfeita. Se o processo A envia uma mensagem a B, existe uma relação causal: o envio ocorreu antes da recepção. Relações de “aconteceu antes” permitem raciocinar sobre ordem parcial sem exigir relógios físicos perfeitamente sincronizados. Relógios lógicos, como Lamport, atribuem contadores que preservam causalidade necessária, embora não revelem tempo real.

Em aplicações simples, não implementaremos um algoritmo de relógio lógico completo, mas o estudante deve reconhecer a diferença entre ordem de chegada, timestamp do produtor e causalidade. Um broker pode entregar eventos em ordem diferente da criação global, e um consumidor precisa decidir se isso importa.

10.9 Consistência e disponibilidade em presença de partições

Quando a rede se divide e dois lados continuam recebendo operações, o sistema precisa escolher que propriedades preservar. Alguns domínios preferem rejeitar alterações sem quorum para proteger consistência; outros aceitam alterações locais e reconciliam posteriormente para manter disponibilidade. Não existe resposta universal: saldo bancário, curtidas em rede social e telemetria podem ter necessidades diferentes.

10.10 Padrões de resiliência combinados

Tabela 19 - Padrões de resiliência e seus trade-offs

Padrão	Problema	Risco de uso incorreto
Timeout	Evitar espera indefinida	Curto demais causa falsos fracassos; longo demais ocupa recursos
Retry	Superar falha transitória	Duplicação e tempestade de tráfego
Backoff + jitter	Espalhar retries no tempo	Aumenta latência de recuperação se exagerado
Circuit breaker	Parar de pressionar dependência falha	Pode bloquear recuperação se política ruim
Bulkhead	Isolar recursos por fluxo	Subutilização e configuração complexa
Fallback	Oferecer resposta degradada	Pode esconder erro ou fornecer dado inadequado

10.11 Idempotency key

Uma estratégia para comandos não idempotentes é o cliente enviar uma chave única por intenção de negócio. O servidor registra a chave e o resultado; se receber a mesma chave novamente, devolve o resultado anterior em vez de repetir o efeito. Isso é comum em pagamentos e criação de recursos. A retenção da chave deve cobrir a janela de retry relevante e sua implementação precisa ser transacional com o efeito protegido.

10.12 Threat modeling simplificado

Antes de escolher controles, identifique ativos, fronteiras de confiança, agentes e possíveis abusos. Pergunte quem pode enviar mensagens, quem pode escutar o canal, quem controla o broker, que dados são sensíveis e o impacto de falsificação, alteração, negação de serviço ou vazamento. O objetivo não é listar todas as vulnerabilidades, mas conectar ameaças concretas a controles verificáveis.